

LAPORAN PRAKTIKUM
ALGORITMA PEMROGRAMAN 2
MODUL 14 – SORTING



NAMA : CESYA KALALUNA PUTRI VALERINE

NIM : 109082500110

KELAS S1IF-13-06

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Pengurutan atau sorting merupakan proses mengatur susunan sekumpulan data acak ke dalam urutan tertentu, baik secara menaik (ascending) dari nilai terkecil ke terbesar, maupun secara menurun (descending) dari nilai terbesar ke terkecil. Proses pengurutan ini sangat penting dalam dunia pemrograman karena data yang rapi dan terstruktur akan jauh lebih mudah dibaca, dikelola, serta mempercepat proses pencarian data spesifik di kemudian hari. Di dalam modul ini, fokus pengurutan dilakukan menggunakan dua metode utama yang sederhana namun efektif, yaitu Selection Sort (pengurutan seleksi) dan Insertion Sort (pengurutan penyisipan).

Metode pertama, yaitu Selection Sort, bekerja dengan cara menyisir seluruh rentang data acak yang tersisa untuk mencari satu nilai paling ekstrim (bisa nilai terkecil atau terbesar tergantung kebutuhan). Setelah nilai ekstrim tersebut ditemukan melalui proses pencarian indeks, nilai tersebut langsung dipindahkan atau ditukar posisinya dengan data yang berada di ujung paling kiri pada barisan data yang belum terurut. Proses pencarian nilai ekstrim dan pertukaran tempat ini terus diulang secara bertahap satu per satu hingga data yang tersisa hanya tinggal satu, sehingga seluruh barisan data otomatis menjadi rapi berurutan.

Metode kedua, yaitu Insertion Sort, memiliki cara kerja yang berbeda karena tidak perlu mencari nilai paling ekstrim terlebih dahulu di awal proses. Ide utama dari algoritma ini mirip seperti saat kita merapikan kartu permainan di tangan, di mana kita mengambil sebuah data yang belum terurut lalu mencari posisi yang tepat pada rangkaian data yang sudah teratur. Saat menyisipkan data baru, data-data lama yang nilainya lebih besar atau lebih kecil akan digeser ke samping (ke kanan) secara berurutan guna membuka satu ruang kosong bagi data baru tersebut. Proses pemilihan data dan penyisipan ini terus diulang untuk setiap elemen data hingga tidak ada lagi data yang tertinggal dalam kondisi acak.

B. Guided

1. Guided1.go

```
package main

import "fmt"

const nMax int = 4321

type arrInt [nMax]int

func insertionSort(T *arrInt, n int) {
    /* I.S. terdefinisi array T yang berisi n bilangan
    bulat
```

```
F.S. array T terurut secara mengecil atau descending
dengan INSERTION SORT */
```

```
var temp, i, j int

i = 1
for i <= n-1 {
    j = i
    temp = T[j]

    for j > 0 && temp > T[j-1] {
        T[j] = T[j-1]
        j = j - 1
    }

    T[j] = temp
    i = i + 1
}

func main() {
    var data arrInt
    var n int

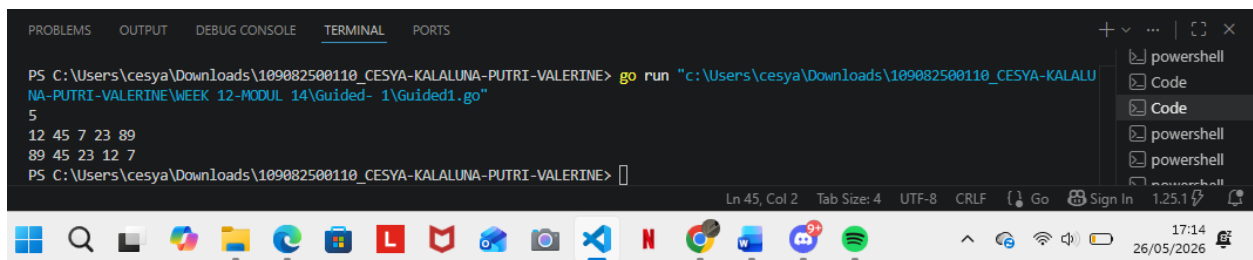
    fmt.Scan(&n)

    for i := 0; i < n; i++ {
        fmt.Scan(&data[i])
    }

    insertionSort(&data, n)

    for i := 0; i < n; i++ {
        fmt.Print(data[i], " ")
    }
}
```

Output :



```
PS C:\Users\cesya\Downloads\109082500110_CESYA-KALALUNA-PUTRI-VALERINE> go run "c:\Users\cesya\Downloads\109082500110_CESYA-KALALUNA-PUTRI-VALERINE\WEEK 12-MODUL 14\Guided- 1\Guided1.go"
5
12 45 7 23 89
89 45 23 12 7
PS C:\Users\cesya\Downloads\109082500110_CESYA-KALALUNA-PUTRI-VALERINE>
```

Penjelasan :

Program ini berfungsi untuk mengurutkan kumpulan bilangan bulat dari yang nilainya paling besar ke yang paling kecil menggunakan metode Insertion Sort. Cara kerjanya di mana program akan membaca jumlah data lalu mengambil angka satu per satu dari urutan kedua hingga terakhir untuk disisipkan ke posisi yang tepat. Di setiap tahapannya, angka yang sedang diperiksa akan dibandingkan dengan angka-angka di sebelah kirinya; jika angka tersebut lebih besar, maka angka di kirinya akan digeser ke kanan untuk memberikan jalan, dan proses ini terus berulang sampai seluruh angka menempati posisi yang benar-benar runtun menurun sebelum akhirnya dicetak ke layar.

2. Guided2.go

```
package main

import "fmt"

const nMax int = 100

type arrString [nMax]string

func insertionSortString(T *arrString, n int) {
    var temp string
    var i, j int

    i = 1
    for i <= n-1 {
        j = i
        temp = T[j]

        for j > 0 && temp < T[j-1] {
            T[j] = T[j-1]
            j = j - 1
        }

        T[j] = temp
        i = i + 1
    }
}

func main() {
    var data arrString
    var n int

    fmt.Scan(&n)

    for i := 0; i < n; i++ {
```

```

        fmt.Scan(&data[i])
    }

    insertionSortString(&data, n)

    for i := 0; i < n; i++ {
        fmt.Print(data[i], " ")
    }
}

```

Output :

```

PS C:\Users\cesya\Downloads\109082500110_CESYA-KALALUNA-PUTRI-VALERINE> go run "c:\Users\cesya\Downloads\109082500110_CESYA-KALALUNA-PUTRI-VALERINE\WEEK 12-MODUL 14\Guided- 2\Guided2.go"
5
Pisang Apel Jeruk Mangga Anggur
Anggur Apel Jeruk Mangga Pisang
PS C:\Users\cesya\Downloads\109082500110_CESYA-KALALUNA-PUTRI-VALERINE>

```

Penjelasan :

Program ini berfungsi untuk mengurutkan kumpulan kata (string) secara alfabetis dari huruf A sampai Z (menaik) menggunakan metode Insertion Sort. Cara kerjanya seperti menyusun kamus atau merapikan kartu nama, di mana program akan membaca jumlah kata lalu mengambil kata satu per satu mulai dari urutan kedua untuk disisipkan ke posisi yang tepat di sebelah kirinya. Di setiap tahapan, kata yang sedang diperiksa akan dibandingkan dengan kata-kata sebelumnya berdasarkan urutan abjad; jika kata tersebut memiliki huruf awalan yang lebih mendekati A dibanding kata di kirinya, maka kata di kirinya akan digeser ke kanan untuk memberikan ruang kosong sampai kata tersebut mapan di posisi abjad yang benar sebelum akhirnya seluruh kata dicetak rapi secara berurutan.

3. Guided3.go

```
package main

import "fmt"

func selectionSortArray(angka *[5]int){
    var idx_min, i, j int

    for i = 0; i < len(angka) - 1; i++ { //loop luar (iterasi
    sortingnya)

        idx_min = i

        for j = i + 1; j < len(angka); j++ { //loop dalam (mencari
        nilai ekstrim)

            if angka[j] <= angka[idx_min] { //sorting terkecil ke
            terbesar

                idx_min = j

            }

        }

        if idx_min != i {

            angka[i], angka[idx_min] = angka[idx_min], angka[i]

        }

    }

}

func main(){

    var arrAngka [5]int

    for i := 0; i < len(arrAngka); i++ {

        fmt.Printf("Masukkan data angka ke-%d : ", i)

        fmt.Scan(&arrAngka[i])

    }

    fmt.Println()

}
```

```

    fmt.Println("=== SEBELUM DISORTING ===")

    for i := 0; i < len(arrAngka); i++ {
        fmt.Print(arrAngka[i], " - ")
    }

    fmt.Println("\n=== SETELAH DISORITNG ===")
    selecetionSortArray(&arrAngka)

    for i := 0; i < len(arrAngka); i++ {
        fmt.Print(arrAngka[i], " - ")
    }
}

```

Output :

```

PS C:\Users\cesya\Downloads\109082500110_CESYA-KALALUNA-PUTRI-VALERINE> go run "c:\Users\cesya\Downloads\109082500110_CESYA-KALALUNA-PUTRI-VALERINE\WEEK 12-MODUL 14\Guided- 3\Guided3.go"
Masukkan data angka ke-0 : 34
Masukkan data angka ke-1 : 12
Masukkan data angka ke-2 : 5
Masukkan data angka ke-3 : 89
Masukkan data angka ke-4 : 1

=== SEBELUM DISORTING ===
34 - 12 - 5 - 89 - 1 -
=== SETELAH DISORITNG ===
1 - 5 - 12 - 34 - 89 -
PS C:\Users\cesya\Downloads\109082500110_CESYA-KALALUNA-PUTRI-VALERINE>

```

Penjelasan :

Program ini berfungsi untuk mengurutkan lima buah bilangan bulat dari yang terkecil ke yang terbesar menggunakan metode Selection Sort. Cara kerjanya dimulai dengan meminta pengguna memasukkan lima angka acak satu per satu ke dalam wadah penyimpanan (array), lalu menampilkannya sebelum proses pengurutan dimulai. Di dalam fungsi pengurutan, program akan menyisir deretan angka tersebut dari depan ke belakang untuk mencari tahu angka mana yang bernilai paling kecil; setelah angka terkecil ditemukan, posisi angka tersebut akan ditukar dengan angka yang berada di garis depan barisan. Proses pencarian angka terkecil ini terus diulang untuk sisa angka lainnya hingga seluruh barisan bergeser dan tersusun rapi secara berurutan, lalu program mencetak hasil akhirnya ke layar dengan pembatas tanda strip (-).

4. Guided4.go

```
package main

import "fmt"

type mahasiswa struct {
    nama, nim string
    IPK        float64
}

func SelectionSortArray(sMHS *[5]mahasiswa) {
    var idx_min, i, j int
    for i = 0; i < len(sMHS)-1; i++ { //loop luar
        idx_min = i
        for j = i + 1; j < len(sMHS); j++ { //loop dalam mencari
            nilai ekstrim
            if sMHS[j].IPK < sMHS[idx_min].IPK { //sorting
                terkecil ke terbesar
                idx_min = j
            }
        }
        if idx_min != i {
            sMHS[i], sMHS[idx_min] = sMHS[idx_min], sMHS[i]
        }
    }
}

func main() {
    var structMHS [5]mahasiswa
```

```

for i := 0; i < len(structMHS); i++ {
    fmt.Printf("--- Masukkan Data Mahasiswa Ke-%d ---\n", i)
    fmt.Print("Nama : ")
    fmt.Scan(&structMHS[i].nama)
    fmt.Print("NIM : ")
    fmt.Scan(&structMHS[i].nim)
    fmt.Print("IPK : ")
    fmt.Scan(&structMHS[i].IPK)
}

fmt.Println()

fmt.Println(" === SEBELUM SORITNG === ")
for i := 0; i < len(structMHS); i++ {
    fmt.Printf("--- Data Mahasiswa Ke-%d ---\n", i)
    fmt.Printf("Nama : %s\n", structMHS[i].nama)
    fmt.Printf("NIM : %s\n", structMHS[i].nim)
    fmt.Printf("IPK : %.2f\n", structMHS[i].IPK)
}

fmt.Println("=====")
fmt.Println()

SelectionSortArray(&structMHS)

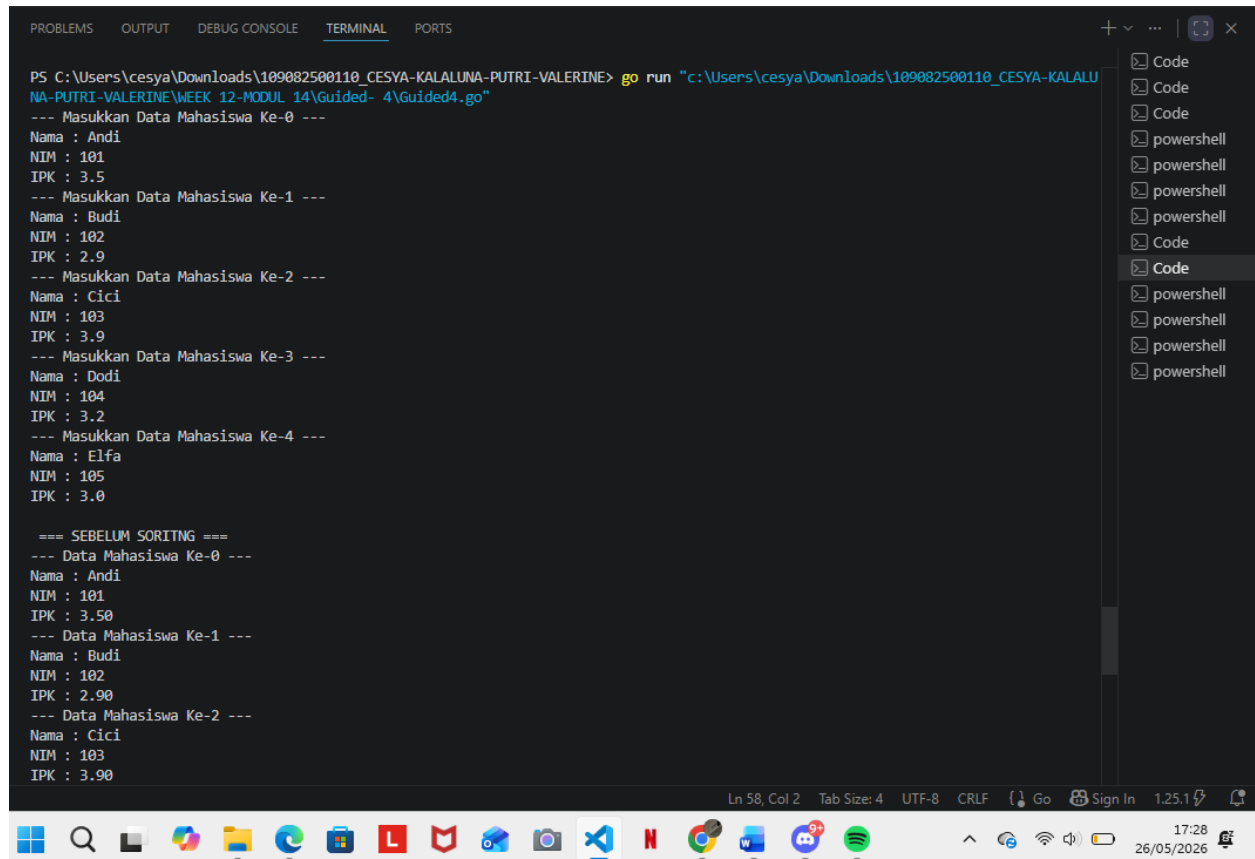
fmt.Println(" === SETELAH SORITNG === ")
for i := 0; i < len(structMHS); i++ {
    fmt.Printf("--- Data Mahasiswa Ke-%d ---\n", i)
    fmt.Printf("Nama : %s\n", structMHS[i].nama)
    fmt.Printf("NIM : %s\n", structMHS[i].nim)
    fmt.Printf("IPK : %.2f\n", structMHS[i].IPK)
}

```

```
        fmt.Println("=====")

        fmt.Println()
    }
}
```

Output :



```
PS C:\Users\cesya\Downloads\109082500110_CESYA-KALALUNA-PUTRI-VALERINE> go run "c:\Users\cesya\Downloads\109082500110_CESYA-KALALUNA-PUTRI-VALERINE\WEEK 12-MODUL 14\Guided- 4\Guided4.go"
--- Masukkan Data Mahasiswa Ke-0 ---
Nama : Andi
NIM : 101
IPK : 3.5
--- Masukkan Data Mahasiswa Ke-1 ---
Nama : Budi
NIM : 102
IPK : 2.9
--- Masukkan Data Mahasiswa Ke-2 ---
Nama : Cici
NIM : 103
IPK : 3.9
--- Masukkan Data Mahasiswa Ke-3 ---
Nama : Dodi
NIM : 104
IPK : 3.2
--- Masukkan Data Mahasiswa Ke-4 ---
Nama : Elfa
NIM : 105
IPK : 3.0

=== SEBELUM SORTING ===
--- Data Mahasiswa Ke-0 ---
Nama : Andi
NIM : 101
IPK : 3.50
--- Data Mahasiswa Ke-1 ---
Nama : Budi
NIM : 102
IPK : 2.90
--- Data Mahasiswa Ke-2 ---
Nama : Cici
NIM : 103
IPK : 3.90
```

```
--- Data Mahasiswa Ke-3 ---  
Nama : Dodi  
NIM : 104  
IPK : 3.20  
--- Data Mahasiswa Ke-4 ---  
Nama : Elfa  
NIM : 105  
IPK : 3.00  
=====
```

```
=== SETELAH SORITNG ===  
--- Data Mahasiswa Ke-0 ---  
Nama : Budi  
NIM : 102  
IPK : 2.90  
--- Data Mahasiswa Ke-1 ---  
Nama : Elfa  
NIM : 105  
IPK : 3.00  
--- Data Mahasiswa Ke-2 ---  
Nama : Dodi  
NIM : 104  
IPK : 3.20  
--- Data Mahasiswa Ke-3 ---  
Nama : Andi  
NIM : 101  
IPK : 3.50  
--- Data Mahasiswa Ke-4 ---  
Nama : Cici  
NIM : 103  
IPK : 3.90  
=====
```

```
PS C:\Users\cesya\Downloads\109082500110_CESYA-KALALUNA-PUTRI-VALERINE>
```

Penjelasan :

Program ini berfungsi untuk mencatat data lima orang mahasiswa beserta nilai IPK mereka, kemudian mengurutkan urutan tampilannya dari yang memiliki IPK terkecil hingga terbesar menggunakan metode Selection Sort. Prosesnya dimulai dengan meminta pengguna mengisi nama, NIM, dan IPK untuk masing-masing mahasiswa ke dalam daftar kelompok data (array struct), lalu menampilkan daftar asli tersebut ke layar. Di dalam proses pengurutan, program akan memeriksa dan menyisir nilai IPK seluruh mahasiswa dari depan ke belakang untuk mencari siapa yang memiliki nilai IPK paling rendah; setelah ditemukan, posisi data mahasiswa tersebut akan ditukar dengan data mahasiswa yang berada di barisan paling depan. Proses pencarian IPK terendah ini terus diulangi untuk posisi-posisi berikutnya sampai semua data mahasiswa bergeser dan tersusun rapi berurutan menaik berdasarkan nilai prestasi mereka.

Unguided

1. Soal Unguided 1

Hercules, preman terkenal seantero ibukota, memiliki kerabat di banyak daerah. Tentunya Hercules sangat suka mengunjungi semua kerabatnya itu. Diberikan masukan nomor rumah dari semua kerabatnya di suatu daerah, buatlah program rumah kerabat yang akan menyusun nomor-nomor rumah kerabatnya secara terurut membesar menggunakan algoritma selection sort. Masukan dimulai dengan sebuah integer n ($0 < n < 1000$), banyaknya daerah kerabat Hercules tinggal. Isi n baris berikutnya selalu dimulai dengan sebuah integer m ($0 < m < 1000000$) yang menyatakan banyaknya rumah kerabat di daerah tersebut, diikuti dengan rangkaian bilangan bulat positif, nomor rumah para kerabat.

Keluaran terdiri dari n baris, yaitu rangkaian rumah kerabatnya terurut membesar di masing masing daerah..

Jawaban :

```
package main

import (
    "fmt"
)

// Fungsi untuk mengurutkan array menggunakan algoritma
Selection Sort
func selectionSort(arr []int) {
    n := len(arr)
    for i := 0; i < n-1; i++ {
        // Cari indeks dengan nilai terkecil di sisa
        array
        minIndex := i
        for j := i + 1; j < n; j++ {
            if arr[j] < arr[minIndex] {
                minIndex = j
            }
        }
        // Tukar nilai terkecil yang ditemukan
        dengan elemen di indeks i
        arr[i], arr[minIndex] = arr[minIndex],
        arr[i]
    }
}

func main() {
    var n int
    // Membaca banyaknya daerah (n)
    if _, err := fmt.Scan(&n); err != nil {
        return
    }
}
```

```

    }

    // Mengolah data untuk setiap daerah
    for i := 0; i < n; i++ {
        var m int
        // Membaca banyaknya kerabat di daerah
        tersebut (m)
        if _, err := fmt.Scan(&m); err != nil {
            break
        }

        // Membuat slice untuk menampung nomor rumah
        sebanyak m elemen
        rumah := make([]int, m)
        for j := 0; j < m; j++ {
            fmt.Scan(&rumah[j])
        }

        // Mengurutkan data menggunakan Selection
        Sort
        selectionSort(rumah)

        // Menampilkan hasil yang sudah terurut
        for k := 0; k < m; k++ {
            if k > 0 {
                fmt.Print(" ")
            }
            fmt.Print(rumah[k])
        }
        fmt.Println()
    }
}

```

Output :

```

PS C:\Users\cesya\Downloads\109082500110_CESYA-KALALUNA-PUTRI-VALERINE> go run "c:\Users\cesya\Downloads\109082500110_CESYA-KALALUNA-PUTRI-VALERINE\WEEK 12-MODUL 14\Unguided- 1\Unguided1.go"
3
5 2 1 7 9 13
1 2 7 9 13
6 189 15 27 39 75 133
15 27 39 75 133 189
3 4 9 1
1 4 9
PS C:\Users\cesya\Downloads\109082500110_CESYA-KALALUNA-PUTRI-VALERINE>

```

Penjelasan :

Program ini dibuat untuk mengurutkan nomor rumah kerabat Hercules secara berurutan dari yang terkecil ke terbesar menggunakan metode Selection Sort. Cara kerjanya diawali dengan membaca jumlah daerah (n), lalu program akan mengulang proses yang sama untuk setiap daerah tersebut dengan membaca jumlah rumah (m) beserta semua nomor rumahnya. Di setiap daerah, fungsi selectionSort akan menyisir deretan nomor rumah untuk mencari angka yang paling kecil, lalu menukarnya ke posisi paling depan, dan proses pencarian ini diulang terus untuk posisi-posisi berikutnya sampai semua nomor rumah rapi berurutan. Setelah selesai diurutkan, program langsung mencetak nomor rumah di daerah tersebut ke layar dengan jeda spasi, lalu berpindah ke daerah selanjutnya hingga seluruh data selesai diproses.

2. Soal Unguided 2

Belakangan diketahui ternyata Hercules itu tidak berani menyeberang jalan, maka selalu diusahakan agar hanya menyeberang jalan sesedikit mungkin, hanya diujung jalan. Karena nomor rumah sisi kiri jalan selalu ganjil dan sisi kanan jalan selalu genap, maka buatlah program kerabat dekat yang akan menampilkan nomor rumah mulai dari nomor yang ganjil lebih dulu terurut membesar dan kemudian menampilkan nomor rumah dengan nomor genap terurut mengecil.

Masukan masih persis sama seperti sebelumnya.

Keluaran terdiri dari n baris, yaitu rangkaian rumah kerabatnya terurut membesar untuk nomor ganjil, diikuti dengan terurut mengecil untuk nomor genap, di masing-masing daerah.

Jawaban :

```
package main
import (
    "fmt"
)

// Fungsi untuk mengurutkan array secara membesar
(Ascending)
func sortAscending(arr []int) {
    n := len(arr)
    for i := 0; i < n-1; i++ {
        minIndex := i
        for j := i + 1; j < n; j++ {
            if arr[j] < arr[minIndex] {
                minIndex = j
            }
        }
    }
}
```

```

        arr[i], arr[minIndex] = arr[minIndex],
arr[i]
    }
}

// Fungsi untuk mengurutkan array secara mengecil
(Descending)
func sortDescending(arr []int) {
    n := len(arr)
    for i := 0; i < n-1; i++ {
        maxIndex := i
        for j := i + 1; j < n; j++ {
            if arr[j] > arr[maxIndex] {
                maxIndex = j
            }
        }
        arr[i], arr[maxIndex] = arr[maxIndex],
arr[i]
    }
}

func main() {
    var n int
    if _, err := fmt.Scan(&n); err != nil {
        return
    }

    for i := 0; i < n; i++ {
        var m int
        if _, err := fmt.Scan(&m); err != nil {
            break
        }

        // Siapkan wadah terpisah untuk menampung
        nomor ganjil dan genap
        var ganjil []int
        var genap []int

        for j := 0; j < m; j++ {
            var nomor int
            fmt.Scan(&nomor)
            // Pisahkan angka berdasarkan
            kondisinya
            if nomor%2 != 0 {
                ganjil = append(ganjil, nomor)
            } else {
                genap = append(genap, nomor)
            }
        }

        // Urutkan angka ganjil dari kecil ke besar

```

```

        sortAscending(ganjil)
        // Urutkan angka genap dari besar ke kecil
        sortDescending(genap)

        // Gabungkan hasil urutan ganjil dan genap
        untuk dicetak
        hasil := append(ganjil, genap...)

        // Cetak seluruh nomor rumah yang sudah rapi
        for k := 0; k < len(hasil); k++ {
            if k > 0 {
                fmt.Print(" ")
            }
            fmt.Print(hasil[k])
        }
        fmt.Println()
    }
}

```

Output :

```

PS C:\Users\cesya\Downloads\109082500110_CESYA-KALALUNA-PUTRI-VALERINE> go run "c:\Users\cesya\Downloads\109082500110_CESYA-KALALUNA-PUTRI-VALERINE\WEEK 12-MODUL 14\Unguided- 2\Unguided2.go"
3
5 2 1 7 9 13
1 7 9 13 2
6 189 15 27 39 75 133
15 27 39 75 133 189
3 4 9 1
1 9 4
PS C:\Users\cesya\Downloads\109082500110_CESYA-KALALUNA-PUTRI-VALERINE>

```

Penjelasan :

Program ini dirancang untuk memisahkan nomor rumah ganjil dan genap di setiap daerah, lalu mengurutkannya dengan aturan yang berbeda demi kenyamanan jalan Hercules. Cara kerjanya adalah saat nomor rumah dibaca satu per satu, program langsung memilahnya: nomor ganjil dimasukkan ke wadah khusus ganjil, dan nomor genap dimasukkan ke wadah khusus genap. Setelah semua nomor di suatu daerah terkumpul, program akan mengurutkan deretan nomor ganjil dari yang terkecil ke terbesar (ascending) dan deretan nomor genap dari yang terbesar ke terkecil (descending). Terakhir, kedua kelompok nomor tersebut digabungkan kembali dengan posisi kelompok ganjil di depan, lalu dicetak secara berurutan dalam satu baris sebelum beralih memproses daerah selanjutnya.

3. Soal unguided 3

Diberikan n data integer positif dalam keadaan terurut membesar dan sebuah integer lain k , apakah bilangan k tersebut ada dalam daftar bilangan yang diberikan? Jika ya, berikan indeksnya, jika tidak sebutkan "TIDAK ADA".

Masukan terdiri dari dua baris. Baris pertama berisi dua buah integer positif, yaitu n dan k . n menyatakan banyaknya data, dimana $1 < n \leq 1000000$. k adalah bilangan yang ingin dicari. Baris kedua berisi n buah data integer positif yang sudah terurut membesar.

Keluaran terdiri dari satu baris saja, yaitu sebuah bilangan yang menyatakan posisi data yang dicari (k) dalam kumpulan data yang diberikan. Posisi data dihitung dimulai dari angka 0. Atau memberikan keluaran "TIDAK ADA" jika data k tersebut tidak ditemukan dalam kumpulan. Program yang dibangun harus menggunakan subprogram dengan mengikuti kerangka yang sudah diberikan berikut ini.

Jawaban:

```
package main

import (
    "fmt"
)

// Fungsi untuk mengurutkan array menggunakan algoritma
Insertion Sort

func insertionSort(arr []int) {
    n := len(arr)
    for i := 1; i < n; i++ {
        key := arr[i]
        j := i - 1
        // Geser elemen yang lebih besar dari key ke posisi
        setelahnya
        for j >= 0 && arr[j] > key {
            arr[j+1] = arr[j]
            j--
        }
    }
}
```

```
        }
        arr[j+1] = key
    }
}

func main() {
    var arr []int
    var val int

    // Membaca masukan terus-menerus sampai ditemukan bilangan
    negatif
    for {
        fmt.Scan(&val)
        if val < 0 {
            break
        }
        arr = append(arr, val)
    }

    // Jika tidak ada data non-negatif yang dimasukkan, program
    selesai
    if len(arr) == 0 {
        return
    }

    // Mengurutkan data dengan Insertion Sort
    insertionSort(arr)
```

```
// Menampilkan data yang sudah terurut
for i := 0; i < len(arr); i++ {
    if i > 0 {
        fmt.Print(" ")
    }
    fmt.Print(arr[i])
}
fmt.Println()

// Memeriksa status jarak (selisih) antar bilangan
if len(arr) < 2 {
    // Jika data hanya 1, otomatis dianggap berjarak tetap
    (misal jarak 0 atau tidak didefinisikan)
    fmt.Println("Data berjarak 0")
    return
}

// Ambil selisih pertama sebagai patokan awal
jarakAwal := arr[1] - arr[0]
berjarakTetap := true

for i := 1; i < len(arr)-1; i++ {
    selisih := arr[i+1] - arr[i]
    if selisih != jarakAwal {
        berjarakTetap = false
        break
    }
}
}
```

```

// Menampilkan status jarak sesuai dengan hasil pemeriksaan
if berjarakTetap {
    fmt.Printf("Data berjarak %d\n", jarakAwal)
} else {
    fmt.Println("Data berjarak tidak tetap")
}
}

```

Output :

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\cesya\Downloads\109082500110_CESYA-KALALUNA-PUTRI-VALERINE> go run "c:\Users\cesya\Downloads\109082500110_CESYA-KALALU
NA-PUTRI-VALERINE\WEEK 12-MODUL 14\Unguided- 3\Unguided3.go"
31 13 25 43 1 7 19 37 -5
1 7 13 19 25 31 37 43
Data berjarak 6
PS C:\Users\cesya\Downloads\109082500110_CESYA-KALALUNA-PUTRI-VALERINE> go run "c:\Users\cesya\Downloads\109082500110_CESYA-KALALU
NA-PUTRI-VALERINE\WEEK 12-MODUL 14\Unguided- 3\Unguided3.go"
4 40 14 8 26 1 38 2 32 -31
1 2 4 8 14 26 32 38 40
Data berjarak tidak tetap
Ln 76, Col 2 Tab Size: 4 UTF-8 CRLF Go Sign In 1.25.1

```

Penjelasan :

Program ini bertugas untuk mengumpulkan angka-angka masukan, mengurutkannya dari yang terkecil ke terbesar menggunakan metode Insertion Sort, dan memeriksa apakah semua angka tersebut memiliki selisih (lompatan) yang sama. Prosesnya dimulai dengan membaca angka satu per satu dan hanya memasukkan angka positif atau nol ke dalam wadah penyimpanan (array), lalu berhenti seketika saat membaca angka negatif. Setelah semua angka positif terkumpul dan diurutkan secara bertahap layaknya menyusun kartu permainan, program akan mencetak deretan angka rapi tersebut ke layar. Di akhir proses, program membandingkan selisih antara tiap angka yang bersebelahan, jika semua selisihnya bernilai sama, maka program mengumumkan jarak konstan tersebut (misalnya, "Data berjarak 6"), namun jika ada satu saja selisih yang berbeda, program akan mengeluarkan status "Data berjarak tidak tetap".

4. Soal Unguided 4

Masukan terdiri dari beberapa baris. Baris pertama adalah bilangan bulat N yang Menyatakan banyak nya data buku yang ada di dalam perpustakaan. N baris berikutnya, masing-masingnya adalah data buku sesuai dengan atribut atau field pada struct. Baris terakhir adalah bilangan bulat yang menyatakan rating buku yang akan dicari.

Keluaran terdiri dari beberapa baris. Baris pertama adalah data buku terfavorit, baris kedua adalah lima judul buku dengan rating tertinggi, selanjutnya baris terakhir adalah data buku yang dicari sesuai rating yang diberikan pada masukan baris terakhir.

Jawaban :

```
package main

import (
    "fmt"
)

const nMax = 7919

// Definisi struct Buku sesuai atribut di gambar
type Buku struct {
    id, judul, penulis, penerbit string
    eksemplar, tahun, rating      int
}

// Menggunakan array dengan kapasitas nMax fix
type DaftarBuku [nMax]Buku

// 1. Prosedur untuk membaca masukan sejumlah n data buku
func DaftarkanBuku(pustaka *DaftarBuku, n *int) {
```

```

    fmt.Scan(n)

    for i := 0; i < *n; i++ {
        fmt.Scan(&pustaka[i].id, &pustaka[i].judul,
&pustaka[i].penulis, &pustaka[i].penerbit,
        &pustaka[i].eksemplar, &pustaka[i].tahun,
&pustaka[i].rating)
    }
}

// 2. Prosedur untuk mencetak buku terfavorit (rating tertinggi)
sebelum diurutkan
func CetakTerfavorit(pustaka DaftarBuku, n int) {
    if n == 0 {
        return
    }
    fav := pustaka[0]
    for i := 1; i < n; i++ {
        if pustaka[i].rating > fav.rating {
            fav = pustaka[i]
        }
    }
    fmt.Printf("%s %s %s %d\n", fav.judul, fav.penulis, fav.penerbit,
fav.tahun)
}

// 3. Prosedur untuk mengurutkan buku menurun berdasarkan rating
menggunakan Insertion Sort
func UrutBuku(pustaka *DaftarBuku, n int) {
    for i := 1; i < n; i++ {
        key := pustaka[i]
        j := i - 1

```

```

        // Karena diurutkan menurun (mengecil), geser elemen yang
        ratingnya lebih kecil

        for j >= 0 && pustaka[j].rating < key.rating {
            pustaka[j+1] = pustaka[j]

            j--
        }

        pustaka[j+1] = key
    }
}

```

// 4. Prosedur untuk mencetak hingga 5 judul buku dengan rating tertinggi

```

func Cetak5Terbaru(pustaka DaftarBuku, n int) {
    limit := 5

    if n < 5 {
        limit = n
    }

    for i := 0; i < limit; i++ {
        if i > 0 {
            fmt.Print(", ")
        }

        fmt.Print(pustaka[i].judul)
    }

    fmt.Println()
}

```

// 5. Prosedur mencari buku dengan rating tertentu menggunakan Pencarian Biner (Binary Search)

```

func CariBuku(pustaka DaftarBuku, n int, r int) {
    left := 0

```

```

right := n - 1

foundIdx := -1

// Binary search pada array yang sudah terurut menurun
for left <= right {
    mid := (left + right) / 2
    if pustaka[mid].rating == r {
        foundIdx = mid
        break // Berhenti jika ditemukan
    } else if pustaka[mid].rating < r {
        right = mid - 1 // Nilai dicari lebih besar, geser ke
kiri karena terurut menurun
    } else {
        left = mid + 1 // Nilai dicari lebih kecil, geser ke
kanan
    }
}

if foundIdx != -1 {
    b := pustaka[foundIdx]
    fmt.Printf("%s %s %s %d %d %d\n", b.judul, b.penulis,
b.penerbit, b.tahun, b.eksemplar, b.rating)
} else {
    fmt.Println("Tidak ada buku dengan rating seperti itu")
}
}

func main() {
    var pustaka DaftarBuku
    var nPustaka int

```

```

var ratingDicari int

// Panggil rangkaian subprogram sesuai skenario tugas
DaftarkanBuku(&pustaka, &nPustaka)
CetakTerfavorit(pustaka, nPustaka)
UrutBuku(&pustaka, nPustaka)
Cetak5Terbaru(pustaka, nPustaka)

// Membaca input terakhir untuk rating yang dicari
fmt.Scan(&ratingDicari)

CariBuku(pustaka, nPustaka, ratingDicari)
}

```

Output :

```

PS C:\Users\cesya\Downloads\109082500110_CESYA-KALALUNA-PUTRI-VALERINE> go run "c:\Users\cesya\Downloads\109082500110_CESYA-KALALUNA-PUTRI-VALERINE\WEEK 12-MODUL 14\Unguided- 4\Unguided4.go"
3
B01 Laskar_Pelangi Andrea_Hirata Bentang_Pustaka 10 2005 9
B02 Bumi_Manusia Pramoedya_Ananta_Toer Lentera_Dipantara 5 1980 10
B03 Negeri_5_Menara A_Fuadi Gramedia 8 2009 8
Bumi_Manusia Pramoedya_Ananta_Toer Lentera_Dipantara 1980
Bumi_Manusia, Laskar_Pelangi, Negeri_5_Menara
9
Laskar_Pelangi Andrea_Hirata Bentang_Pustaka 2005 10 9

```

Penjelasan :

Program ini dirancang untuk mengelola data buku perpustakaan terstruktur dengan menyimpan informasi detail buku ke dalam kumpulan tipe data (array struct). Alur kerjanya dimulai dengan mencatat semua buku yang dimasukkan, lalu mencari dan menampilkan informasi buku terfavorit yang memiliki rating paling tinggi. Selanjutnya, program menyusun ulang seluruh daftar buku agar urut menurun dari rating terbesar ke terkecil menggunakan metode Insertion Sort, sehingga program bisa langsung mengambil dan menampilkan lima judul buku teratas di baris berikutnya. Terakhir, program menggunakan metode pencarian belah dua (Binary Search) yang efisien untuk melacak keberadaan buku tertentu sesuai target rating yang diminta oleh pengguna di baris paling akhir.

C. Kesimpulan

Rangkaian program di atas dirancang untuk membantu kita memahami bagaimana komputer mengolah, menyusun, dan mencari data secara rapi melalui konsep algoritma dasar. Dimulai dari penggunaan Selection Sort dan Insertion Sort untuk mengurutkan angka (seperti nomor rumah) atau data kompleks (seperti daftar buku perpustakaan) baik dari yang terkecil maupun terbesar, hingga penerapan teknik Binary Search untuk menemukan data spesifik dengan waktu yang sangat cepat. Melalui program-program ini, kita bisa melihat bahwa dengan memilah data ke dalam wadah yang tepat (seperti memisahkan angka ganjil-genap) dan menyusunnya secara teratur, komputer dapat menyajikan informasi yang rapi, memeriksa pola keteraturan jarak antar-angka, serta menampilkan hasil pencarian data secara akurat dan efisien.